ENGRAM // COGNITIVE ARCHITECTURE

ENGRAM Developer Guide

DEVELOPER GUIDE

Architecture, data model, services, and workflows for engineers building on the ENGRAM / PYRI framework.

PYRI — AI Persona Framework

Version 1.0 • June 26, 2026 • Engineering

Audience: engineers. Pair this guide with the README for quick-start commands.

1. Introduction

ENGRAM is a TypeScript monorepo (pnpm workspaces) implementing an AI persona framework: a React + Vite dashboard, an Express API, and a PostgreSQL database, tied together by a contract-first OpenAPI pipeline. This guide explains how the pieces fit and how to work on them safely.

> **Mental model**
> The API contract and the database schema are the two sources of truth. Almost every change starts in one of them and flows outward through code generation or a schema push.

2. Technology Stack

`Runtime / language`	Node.js 24, TypeScript 5.9, ES modules across packages.
`Package manager`	pnpm workspaces with a pinned catalog and a minimum-release-age supply-chain guard.
`API`	Express 5 with typed, Zod-validated routes and structured request logging.
`Database`	PostgreSQL with the Drizzle ORM; schema defined in code.
`Validation`	Zod (zod/v4) and drizzle-zod generated insert schemas.
`API codegen`	Orval generates Zod schemas and React Query hooks from the OpenAPI spec.
`Frontend`	React + Vite, Tailwind CSS v4, shadcn/ui, wouter router, TanStack Query, framer-motion, recharts, lucide-react.
`Fonts`	Self-hosted via @fontsource (Inter, Rajdhani, JetBrains Mono) — no CDN dependency.
`Build`	esbuild bundles the server; Vite builds the frontend.

3. Repository Layout

The workspace separates deployable artifacts from shared libraries:

```
artifacts/
  engram/           # React + Vite dashboard (the UI)
  api-server/       # Express API + autonomy engine
  mockup-sandbox/   # component preview sandbox
lib/
  api-spec/         # OpenAPI source of truth + codegen config
  api-zod/          # generated Zod schemas
  api-client-react/# generated React Query hooks + query keys
  db/               # Drizzle schema + client (source of truth for data)
  integrations*/    # optional provider integration helpers
scripts/            # seeding + utility scripts (and this doc generator)
pnpm-workspace.yaml tsconfig.base.json tsconfig.json package.json
```

- **lib/* are composite** they emit declarations via project references and are built with the root solution config.
- **artifacts/* and scripts are leaf packages** checked with tsc --noEmit; they must not import each other — share code through a lib instead.

4. Contract-First Workflow

The OpenAPI document is the single definition of the HTTP surface. Code generation produces matching server schemas and client hooks so the two sides cannot drift.

```
pnpm --filter @workspace/api-spec run codegen
```

- Edit the spec in lib/api-spec, then run codegen.
- Server routes validate inputs/outputs with the generated Zod schemas.
- The frontend consumes generated hooks and the get...QueryKey helpers for cache management.

> **Important**
> Do not change the OpenAPI info.title casually — it controls generated filenames. Always re-run codegen after editing the spec, and run a libs typecheck before checking the leaf artifacts.

5. Data Model

All tables are defined in lib/db and re-exported from a single barrel. Reference tables hold static vocabulary; persona tables hold both design-time config and live runtime state.

5.1 Persona tables

`engrams`	A persona: identity (slug, name, title, symbol, origin), voice profile, emotional baseline, environment anchor, distilled memory seed, guardrails, drives, focus themes, plus autonomy config (enabled, tick cadence, initiation threshold) and live state (drive pressure, last-tick / last-transmission / backoff timestamps, current mood, chat-active flag).

`engram_transmissions`	Self-initiated messages an engram emits autonomously. Records kind (idle vs. outreach), originating drive, content, mood, and importance / confidence / novelty / overall scores, plus delivered/ seen flags.
`engram_inquiries`	A log of probing or developing an engram: kind (probe/develop), question, response, and for develop an applied config delta.

5.2 Conversation & reference tables

`conversations / messages`	Persisted chat threads; a conversation carries a mode, optional persona name, optional custom engram, and an optional engramId link.
`expressions`	The emotive micro-expression library: glyph, name, family, eyes, mouth, optional gesture, valence, arousal, an intimacy integer, optional cognitive role, and notes.
`personality / memories / journal / personas / beliefs / evolution / initiative / hiero`	Reference and content tables backing the corresponding dashboard pages.

6. Backend Services

The API server hosts one route module per domain (personality, memories, journal, personas, beliefs, evolution, initiative, hiero, expressions, engrams, stats, health, and the chat endpoint). Prompt construction and persona generation live in dedicated library modules. The autonomy engine is started when the server boots and stopped on shutdown.

> **Logging convention**
> Never use console.log in server code. Use the request logger inside handlers and the shared singleton logger elsewhere.

7. The Autonomy Engine

A single in-process ticker drives persona autonomy. On each wake it accrues per-drive 'pressure' from elapsed time only — there is no model call in the accrual path. Pressure is jittered and capped per drive; each drive's 'charge' is its pressure times its weight. When the top charge crosses the engram's initiation threshold, the engine asks the model for exactly one transmission, persists it, resets that drive, and bleeds the rest so it does not immediately refire.

Cost and cadence are bounded by tunable constants:

`Global tick`	The engine wakes roughly every 20 seconds.
`Per-engram cooldown`	At least 90 seconds between an engram's transmissions.
`Rate caps`	Up to 10 transmissions per rolling hour and 60 per rolling day, per engram.

`Elapsed cap`	Accrual is capped at 600 seconds to absorb downtime or clock jumps.
`Error backoff`	After a failure an engram is skipped for ~120 seconds; the backoff is persisted.

Because live state (drive pressure, timestamps, backoff) is stored in the database, autonomy resumes coherently after a restart rather than resetting.

8. The LLM Provider Seam

Every model call goes through one OpenAI-compatible client created in a single module. The endpoint, key, and model name are resolved from environment variables with a defined precedence, so the same code targets a hosted model or a local runtime with no code changes.

Resolution order (first defined wins):

- **Explicit override:** LLM_BASE_URL, LLM_API_KEY, LLM_MODEL (use these to point at a local server).
- **Cloud fallback:** AI_INTEGRATIONS_OPENAI_BASE_URL / ..._API_KEY, with a default model name.

If no endpoint is configured the module throws a clear startup error rather than failing silently. The cloud integration package is intentionally not imported on the chat or transmission path, so a fresh local deploy without cloud variables still starts.

9. Prompt Building & Safety

Prompts are assembled in code from persona configuration and relevant state. Safety limits are hard-coded rather than expressed in prompt text. In particular, the expression layer's intimacy axis is sanitized out before any prompt is built — only a clamped, neutral form survives, and free-text notes never reach the model. Persona self-development (the inquiry 'develop' mode) can mutate only a small, explicitly bounded set of fields (such as mood, threshold, cadence, focus themes, drive weights, and added facts), returned as a structured delta.

> **Important**
> Keep all hard safety in code. Do not move sanitization or bounds into stored persona data or prompt strings, where adversarial input could bypass them.

10. Frontend Architecture

The dashboard is a single-page React app. Pages live under a pages directory (Hub, Chat, Environment, Inquiry, Personality, Memory, Journal, Personas, Hiero-Code, Beliefs, Evolution, Analytics, and Initiative). Routing is handled by wouter; navigation is defined in the shared layout. Server state uses the generated React Query hooks; UI primitives come from shadcn/ui; styling is Tailwind v4 with a cyberpunk cyan theme.

- **Responsive layout.** Below the medium breakpoint the desktop sidebar is replaced by a top bar with a hamburger that opens navigation in a slide-in panel; content reflows to full width.
- **Chat layout.** The conversation list is a permanent column on desktop and a slide-in panel on phones, chosen via a viewport hook so a shared-state dialog is never mounted twice.

> **Responsive gotcha**
> When making a page responsive where the subtree contains a portaled, shared-state dialog, branch desktop/mobile with the viewport hook — do not render two CSS-toggled copies, or the dialog mounts twice.

11. Environment & Configuration

`DATABASE_URL`	Required. PostgreSQL connection string.
`LLM_BASE_URL`	Optional. OpenAI-compatible endpoint (e.g. a local server). Overrides the cloud default.
`LLM_API_KEY`	Optional. Key for the endpoint; local servers usually accept any non-empty placeholder.
`LLM_MODEL`	Optional. Model name to request. Defaults to the cloud model name, so local deployments should set this to a model the local server actually hosts.
`LLM_VISION_MODEL / LLM_TRANSCRIBE_MODEL`	Optional. Models for image/video vision and audio/video transcription; needed only for those media features.
`AI_INTEGRATIONS_OPENAI_*`	Optional cloud fallback used when the LLM_* overrides are absent.
`PORT`	Port the API listens on. Provided by the platform workflow on Replit; set in .env for local runs (default 5000).
`BASE_PATH`	Base path the frontend is built against ("/" for local single-port runs).
`WEB_DIST`	Optional. Absolute path to the built dashboard (artifacts/engram/dist/public). When set, the API also serves the dashboard so the whole app runs on one port. Left unset on Replit, where the web is a separate static artifact.

12. Local / Offline Operation

12.1 Pointing at a local model

To run with no cloud dependency, point the provider seam at any OpenAI-compatible server (Ollama, LM Studio, llama.cpp, vLLM):

```
# example: Ollama
export LLM_BASE_URL=http://localhost:11434/v1
export LLM_MODEL=llama3.1
export LLM_API_KEY=local   # any non-empty placeholder
```

With these set, both interactive chat and autonomous transmissions run entirely against the local model. Nothing else in the code needs to change.

12.2 One-command single-port run

The repository ships cross-platform launch scripts (scripts/local/) that build the dashboard and the API and run them together on a single port, with the API serving the built frontend. The first run also installs dependencies, pushes the schema, and seeds reference data.

```
# Linux / macOS
./scripts/local/start.sh

# Windows (PowerShell)
.\scripts\local\start.ps1
```

Configuration is read from a .env file (copied from .env.example on first run); the scripts load it into the environment so the schema push, the Vite build, and the server all see it. Single-port serving is gated on WEB_DIST: when it points at the built dashboard the API serves both the UI and /api. Because the frontend calls the API with same-origin relative paths, one Express process needs no proxy or URL rewriting. On Replit WEB_DIST is unset, so the API serves only /api and the web stays a separate static artifact — behavior there is unchanged.

After the initial dependency download and model pull, the app runs with no outbound network: fonts are self-hosted, the frontend talks to the local API, and the API talks to the local model.

> **Important**
> Video media perception additionally needs ffmpeg + ffprobe on PATH; image/video vision and audio/video transcription need a model that supports those modalities. A text-only local model still handles chat and autonomous transmissions.

13. Build, Run & Common Commands

`Run fully local (one port)`	./scripts/local/start.sh (Windows: .\scripts\local\start.ps1)
`Run the API (dev)`	pnpm --filter @workspace/api-server run dev
`Run the dashboard (dev)`	pnpm --filter @workspace/engram run dev
`Full typecheck`	pnpm run typecheck
`Build everything`	pnpm run build
`Regenerate API artifacts`	pnpm --filter @workspace/api-spec run codegen
`Push schema (dev only)`	pnpm --filter @workspace/db run push

`Seed expressions`	pnpm --filter @workspace/scripts run seed:expressions
`Seed engrams`	pnpm --filter @workspace/scripts run seed:engrams
`Seed hub / spaces`	pnpm --filter @workspace/scripts run seed:hub
`Regenerate docs (PDFs)`	node scripts/gen/generate-docs.mjs

> **Important**
> Do not run pnpm dev at the workspace root. Apps run via platform workflows that provide PORT and BASE_PATH. Verify a package with its typecheck script rather than build when running ad hoc from the shell.

14. Database & Seeding

A fresh database created by a schema push leaves reference tables empty — there is no auto-seed. Run the seeding scripts to populate the expression library and the default engrams. The expression seed is idempotent, so it is safe to re-run. Add a matching seed script whenever you introduce a new reference table.

15. Coding Conventions & Gotchas

- Keep changes in the right source of truth: spec for the HTTP surface, schema for the data shape; then regenerate or push.
- Run a libs typecheck before leaf typechecks; a missing db export is usually stale lib declarations, not a bad import.
- Use the generated query-key helpers to invalidate caches after mutations.
- Fonts are self-hosted and imported in the app entry point — do not re-add CDN font links.
- Persona expression behavior must remain platonic; the intimacy axis is sanitized before prompts regardless of stored values.

16. Troubleshooting

`Server won't start: no LLM endpoint`	Set LLM_BASE_URL (local) or the cloud integration variables.
`Empty reference tables after push`	Run the seed scripts; the push does not seed data.
`Type errors after editing a lib`	Run the libs typecheck first to refresh declarations.
`Generated hooks out of date`	Re-run codegen after any spec change.
`Preview is blank in the platform`	Ensure the service binds the assigned PORT and the dev server allows proxied hosts.
`Local dashboard 404s on /api`	Confirm WEB_DIST points at a built dashboard and the API and UI share the same port.

`Video media job fails`	Install ffmpeg + ffprobe on PATH; text/image/audio modalities are unaffected.